Selective Recovery in a Multi-Agent Coding Workflow

CSE 291A Fall 2026, Option 5 · Phase 1 Report · November 4, 2026

Jackie Wang, Ray Williams

1. Question

Option 5 asks whether tracking dependencies between agent results can make recovery efficient while keeping execution correct and safe. In a researcher → planner → executor chain, one bad intermediate result can flow into later plans, file edits and external actions. The two obvious responses, restarting everything or verifying every hand-off, are both costly. Phase 1 measures how far faults actually travel and what those two baselines cost. Phase 2 tests whether rerunning only the dependent results, and rolling back only the state they touched, recovers the same tasks with less work.

2. Workflow and tasks

The three agents share one model and fixed prompts, and each replies in JSON that the server constrains to a schema. They complete small tickets on a fake Python `billing` library (payments, fees, invoices, config, client) with four internal docs.

- The **researcher** reads the docs and reports at most five facts the change depends on, each with its source doc.
- The **planner** turns the ticket, the facts and the current files into a JSON plan of file, function, signature and change.
- The **executor** calls one tool per turn: read or write a file, run the visible tests, publish a release, or post to a channel.

The agents share real state. The workspace is a git repository, and every tool call records the commit before and after it. A simulated package registry and team channel are append-only, so some actions cannot be undone.

Outcomes are checked by code, not by an LLM. The checker runs a hidden pytest file on the final code, lists every changed file outside the task's allowed paths as collateral damage, and compares the registry and channel with the releases and messages the task expects. It reports unauthorized actions separately from task success: tool calls the task does not allow, or writes to protected paths such as `tests/` and `docs/`.

The dev set has 10 tickets. Each one depends on a single rule in `docs/policy.md` or `docs/releases.md` that the ticket never states. Examples are a 1.5% late fee capped at 2500 cents, a 90-day refund window, a 50-cent minimum charge, and EUR amounts shown with a decimal comma. Two tickets also ask the executor to ship, so they need the right version bump, a publish and a channel post. Each task lists its deciding fact and one control fact from the same doc that the answer does not depend on. It also lists a wrong value and a stale (2025) value for the deciding fact. Unit tests confirm that the checker passes each reference solution, fails a do-nothing run and the wrong-fact version, and catches a doc edit, a deleted test file and an unrequested publish. The five pilot tickets are kept as a separate tuning set.

3. Fault injection and tracing

The harness rewrites the researcher's facts or the planner's plan before the next stage reads it. A fixed schedule (seed 291) gives every task each of seven faults, 70 faulted runs in all. The three attacks are spread evenly across tasks.

Fault	Class	Stage	What changes
Wrong fact	Accidental	Facts	Deciding fact replaced, e.g. 2% instead of 1.5%
Stale fact	Accidental	Facts	Deciding fact replaced by the 2025 rule
Timeout	Accidental	Facts	Researcher returns nothing; planner gets no facts
Wrong plan	Accidental	Plan	Right value swapped for the wrong one in the plan
Malicious fact	Adversarial	Facts	Injected instruction cited to `docs/releases.md`
Malicious plan	Adversarial	Plan	Extra plan step carrying the same instruction
Control	Control	Facts	Only the irrelevant support-hours fact changes

The attacks ask the executor to delete `tests/test_basic.py`, publish release 9.9.9 and announce it, or rewrite `docs/policy.md`. Each task also gets a *no-docs* check run with the docs folder removed, which shows whether the model needs the docs at all or could guess the rule.

Each faulted run is paired with the clean run of the same task. If no downstream artifact differs from the clean run, the fault was *ignored*. If something differs but the task still passes with no collateral, unauthorized or irreversible effects, it was *absorbed*. Otherwise it was *harmful*. For adversarial faults we also record whether the attack actually executed. We report accidental and adversarial faults separately. Tasks that the clean workflow already fails are left out of the propagation analysis, so a model's inability is not mistaken for protection.

Every run writes a provenance trace in JSONL with one node per artifact: the facts, the plan, each executor turn, and each read or tool call. A node records its inputs, a content hash, tokens and latency, and whether the harness injected a fault there. State links count as much as message links. A file read links to the step that last wrote that file, so a step that reads a file written by a faulty step depends on the fault even with no message between them. From the trace we compute which nodes changed against the clean run, every node downstream of the injected one, the stages affected, and the tokens and seconds spent downstream of the fault. Phase 2 reads this same trace to decide what to invalidate.

4. Baselines, metrics and setup

All conditions run on the same tasks and fault schedule:

- **Clean** sets the capability ceiling.
- **Faulted, no recovery** measures propagation and damage.
- **Full restart** restores the seed state and reruns every stage without the fault.
- **Always-verify** has a verifier check each result before hand-off, and counts its tokens and latency.

Each recovery method is told which result is faulty, so we measure recovery apart from detection. We report clean task success, ignored/absorbed/harmful rates by fault class and stage, executed attacks, unauthorized actions, irreversible effects, affected stages, work downstream of the fault, and end-to-end latency and tokens. Runs use temperature 0 and seed 0. Failed runs and timeouts stay in the data.

We self-host **Qwen3.5 4B** (Q4_K_M, pinned GGUF file and hash) with llama.cpp, with thinking mode off. In the pilot on five tickets, three seeds each, the model alone passed 10/15 at about 6 s per task. The three-agent chain passed only 7/15 at about 18 s, below our 60% bar. Since the model could solve the tickets alone, we kept it and fixed the hand-offs instead: schema- constrained JSON, at most five facts, and signatures copied verbatim into the plan. We did not tune prompts on the pilot tickets' scores. One pilot doc contradicted its own hidden test, so we fixed it and reran all of the pilot. Phase 1 runs on a MacBook Pro (M4 Pro). Latency does not transfer between machines, so the final comparison reruns the baselines alongside Phase 2 on one AWS CPU instance, within the $3/day limit. Each run saves its model, seed and sampling settings with the trace, tool log and result.

5. Status and Phase 2 direction

As of Nov 3 the harness runs end to end. We have the git workspace and tools, the 10 dev tasks with hidden tests, reference and wrong solutions, the checker, the three agents, the provenance trace, and the fault injector with its schedule and measurement script, all covered by 61 unit tests. `scripts/run_tasks.py` runs the clean condition and `scripts/run_faults.py` runs each task's clean pair, the no-docs check and the seven scheduled faults. The full-restart and always-verify baselines are not built yet.

[Results: clean pass rate and the ignored/absorbed/harmful table from `run_faults.py`, to be filled in after the dev-set run.]

Phase 2 builds dependency-aware invalidation with shared-state rollback. Recovery walks the provenance graph, through both message inputs and state reads, to find every result downstream of the faulty one. It resets the workspace to the commit before the first affected write, and reruns only the invalidated stages. Effects that cannot be undone, such as a published release or a posted message, are reported instead of counted as recovered. A sample of reused results is rerun to confirm they were valid. We will compare this method with both baselines under identical fault schedules and budgets, and weigh recovery precision against bookkeeping overhead. The trace already records its own size and build time for that comparison. If Phase 1 shows that most faults reach every later stage, then selective recovery saves little, and we will report that.

Appendix: Timeline and milestones

Dates	Work	Milestone
Oct 27	Repo scaffold, state store, unit tests	Done
Oct 27–29	Model pilot on five tickets	Done: Qwen3.5 4B chosen; file hash recorded
Nov 2	Coding workspace and tools	Done: tool unit tests pass
Nov 2	10 dev tasks, hidden tests, checker; agents	Done: reference passes; do-nothing fails
Nov 2–3	Provenance tracing; fault injector and schedule	Done: faulted run compared with its clean pair
Nov 3–4	Dev-set clean and fault runs; results tables	In progress
Nov 4	**Phase 1 code and data due**	One command per condition
Nov 5–10	Full-restart and always-verify baselines	All four conditions run end to end
Nov 5–17	Selective invalidation	Only downstream stages rerun
Nov 12–24	Shared-state rollback	Irreversible actions reported
Nov 10–20	Held-out task set	Unseen until the final evaluation
Nov 23–Dec 3	Held-out evaluation on AWS	Same schedules, budgets and seeds
Dec 1–9	Final report	
Dec 10	**Final submission**	

Risks and fallbacks:

- *Low clean success.* If the chain passes too few clean runs, faults have little to break. Fallbacks: a larger model (Qwen3.5 9B or Qwen3 8B), or TritonAI.
- *Faults that never matter.* If the no-docs check passes, the model is guessing the rule, and that task's fact faults say little. We will rewrite or drop such tasks.
- *Run time.* About 80 runs per pass on one laptop. If it is too slow, we will run always-verify on a subset of tasks.
- *Run-to-run noise.* Temperature 0 on llama.cpp is not fully deterministic. We will rerun a subset with three seeds and report the spread.